```
-- ================================================================================
-- PROJECT: IndiaKart E-Commerce Analytics — End-to-End Analytics Pipeline
-- PLATFORM: MySQL Workbench 8.0+
-- DATABASE: indiakart_db
-- DATASET: 24 Months Operations (June 2023 - June 2025)
-- TOTAL RECORDS: ~222,000 Records across 8 Interconnected Relational Tables
-- ================================================================================

-- --------------------------------------------------------------------------------
-- INITIAL ENVIRONMENT & DATABASE SETUP
-- --------------------------------------------------------------------------------
CREATE DATABASE IF NOT EXISTS indiakart_db;
USE indiakart_db;

SET NAMES utf8mb4;
SET FOREIGN_KEY_CHECKS = 0;
SET SQL_MODE = 'NO_AUTO_VALUE_ON_ZERO,STRICT_TRANS_TABLES,NO_ENGINE_SUBSTITUTION';

-- --------------------------------------------------------------------------------
-- SECTION 1: DDL — TABLE SCHEMA DEFINITIONS (8 RELATIONAL TABLES)
-- --------------------------------------------------------------------------------

-- 1. Table: suppliers
DROP TABLE IF EXISTS suppliers;
CREATE TABLE suppliers (
    supplier_id         VARCHAR(10)     NOT NULL,
    supplier_name       VARCHAR(100)    NOT NULL,
    contact_person      VARCHAR(80)     NULL,
    email               VARCHAR(120)    NULL,
    phone               VARCHAR(15)     NOT NULL,
    city                VARCHAR(50)     NULL,
    state               VARCHAR(50)     NULL,
    pincode             VARCHAR(10)     NULL,
    category            VARCHAR(50)     NULL,
    gstin               VARCHAR(20)     NULL,
    payment_terms_days  INT             DEFAULT 30,
    rating              DECIMAL(3,1)    DEFAULT 4.0,
    created_date        DATE            NULL,
    is_active           TINYINT(1)      DEFAULT 1,
    PRIMARY KEY (supplier_id),
    INDEX idx_suppliers_state (state),
    INDEX idx_suppliers_category (category)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 2. Table: products
DROP TABLE IF EXISTS products;
CREATE TABLE products (
    product_id          VARCHAR(10)     NOT NULL,
    product_name        VARCHAR(150)    NOT NULL,
    category            VARCHAR(50)     NOT NULL,
    subcategory         VARCHAR(80)     NULL,
    brand               VARCHAR(80)     NULL,
    sku                 VARCHAR(30)     NOT NULL,
    mrp                 DECIMAL(10,2)   NOT NULL,
    selling_price       DECIMAL(10,2)   NOT NULL,
    cost_price          DECIMAL(10,2)   NULL,
    gst_rate            INT             DEFAULT 18,
    hsn_code            VARCHAR(10)     NULL,
    weight_grams        INT             NULL,
    supplier_id         VARCHAR(10)     NULL,
    rating              DECIMAL(3,1)    DEFAULT 4.0,
    review_count        INT             DEFAULT 0,
```

```
    is_active           TINYINT(1)      DEFAULT 1,
    launch_date         DATE            NULL,
    PRIMARY KEY (product_id),
    UNIQUE KEY uq_products_sku (sku),
    INDEX idx_products_category (category),
    INDEX idx_products_brand (brand),
    INDEX idx_products_supplier (supplier_id),
    CONSTRAINT fk_products_supplier FOREIGN KEY (supplier_id) REFERENCES suppliers (supplier_id) ON DELET
    E SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 3. Table: inventory
DROP TABLE IF EXISTS inventory;
CREATE TABLE inventory (
    inventory_id            VARCHAR(10)     NOT NULL,
    product_id              VARCHAR(10)     NOT NULL,
    warehouse_location      VARCHAR(50)     NULL,
    quantity_available      INT             DEFAULT 0,
    quantity_reserved       INT             DEFAULT 0,
    reorder_level           INT             DEFAULT 50,
    reorder_quantity        INT             DEFAULT 100,
    last_restocked_date     DATE            NULL,
    unit_cost               DECIMAL(10,2)   NULL,
    total_inventory_value   DECIMAL(12,2)   NULL,
    status                  VARCHAR(20)     DEFAULT 'In Stock',
    PRIMARY KEY (inventory_id),
    INDEX idx_inventory_product (product_id),
    INDEX idx_inventory_status (status),
    INDEX idx_inventory_warehouse (warehouse_location),
    CONSTRAINT fk_inventory_product FOREIGN KEY (product_id) REFERENCES products (product_id) ON DELETE C
    ASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 4. Table: customers
DROP TABLE IF EXISTS customers;
CREATE TABLE customers (
    customer_id         VARCHAR(12)     NOT NULL,
    first_name          VARCHAR(50)     NOT NULL,
    last_name           VARCHAR(50)     NOT NULL,
    email               VARCHAR(120)    NULL,
    phone               VARCHAR(15)     NULL,
    city                VARCHAR(50)     NULL,
    state               VARCHAR(50)     NULL,
    pincode             VARCHAR(10)     NULL,
    gender              VARCHAR(10)     NULL,
    age                 INT             NULL,
    segment             VARCHAR(20)     DEFAULT 'Regular',
    registration_date   DATE            NULL,
    last_login_date     DATE            NULL,
    is_verified         TINYINT(1)      DEFAULT 0,
    is_active           TINYINT(1)      DEFAULT 1,
    total_orders        INT             DEFAULT 0,
    total_spent         DECIMAL(12,2)   DEFAULT 0.00,
    PRIMARY KEY (customer_id),
    INDEX idx_customers_segment (segment),
    INDEX idx_customers_state (state),
    INDEX idx_customers_city (city)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 5. Table: orders
DROP TABLE IF EXISTS orders;
CREATE TABLE orders (
```

```
    order_id            VARCHAR(12)     NOT NULL,
    customer_id         VARCHAR(12)     NOT NULL,
    order_date          DATE            NOT NULL,
    order_time          TIME            NULL,
    status              VARCHAR(20)     NOT NULL,
    city                VARCHAR(50)     NULL,
    state               VARCHAR(50)     NULL,
    pincode             VARCHAR(10)     NULL,
    total_amount        DECIMAL(12,2)   NULL,
    gst_amount          DECIMAL(10,2)   NULL,
    shipping_charge     DECIMAL(8,2)    DEFAULT 0.00,
    discount_amount     DECIMAL(10,2)   DEFAULT 0.00,
    final_amount        DECIMAL(12,2)   NOT NULL,
    payment_method      VARCHAR(30)     NULL,
    shipping_partner    VARCHAR(50)     NULL,
    tracking_id         VARCHAR(30)     NULL,
    delivered_date      DATE            NULL,
    is_cod              TINYINT(1)      DEFAULT 0,
    channel             VARCHAR(20)     DEFAULT 'App',
    PRIMARY KEY (order_id),
    INDEX idx_orders_customer (customer_id),
    INDEX idx_orders_date (order_date),
    INDEX idx_orders_status (status),
    INDEX idx_orders_state (state),
    INDEX idx_orders_channel (channel),
    CONSTRAINT fk_orders_customer FOREIGN KEY (customer_id) REFERENCES customers (customer_id) ON DELETE
    CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 6. Table: order_items
DROP TABLE IF EXISTS order_items;
CREATE TABLE order_items (
    item_id             VARCHAR(12)     NOT NULL,
    order_id            VARCHAR(12)     NOT NULL,
    product_id          VARCHAR(10)     NOT NULL,
    product_name        VARCHAR(150)    NULL,
    category            VARCHAR(50)     NULL,
    quantity            INT             NOT NULL DEFAULT 1,
    unit_price          DECIMAL(10,2)   NOT NULL,
    gst_rate            INT             DEFAULT 18,
    gst_amount          DECIMAL(10,2)   NULL,
    discount_amount     DECIMAL(10,2)   DEFAULT 0.00,
    total_price         DECIMAL(12,2)   NOT NULL,
    PRIMARY KEY (item_id),
    INDEX idx_order_items_order (order_id),
    INDEX idx_order_items_product (product_id),
    INDEX idx_order_items_category (category),
    CONSTRAINT fk_order_items_order FOREIGN KEY (order_id) REFERENCES orders (order_id) ON DELETE CASCADE
     ON UPDATE CASCADE,
    CONSTRAINT fk_order_items_product FOREIGN KEY (product_id) REFERENCES products (product_id) ON DELETE
     CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 7. Table: payments
DROP TABLE IF EXISTS payments;
CREATE TABLE payments (
    payment_id          VARCHAR(12)     NOT NULL,
    order_id            VARCHAR(12)     NOT NULL,
    customer_id         VARCHAR(12)     NOT NULL,
    payment_date        DATE            NOT NULL,
    payment_time        TIME            NULL,
    payment_method      VARCHAR(30)     NULL,
```

```
    amount              DECIMAL(12,2)   NOT NULL,
    status              VARCHAR(20)     DEFAULT 'Success',
    transaction_id      VARCHAR(30)     NULL,
    bank_name           VARCHAR(60)     NULL,
    gateway             VARCHAR(40)     NULL,
    refund_amount       DECIMAL(10,2)   DEFAULT 0.00,
    refund_date         DATE            NULL,
    PRIMARY KEY (payment_id),
    UNIQUE KEY uq_payments_transaction (transaction_id),
    INDEX idx_payments_order (order_id),
    INDEX idx_payments_customer (customer_id),
    INDEX idx_payments_status (status),
    INDEX idx_payments_gateway (gateway),
    CONSTRAINT fk_payments_order FOREIGN KEY (order_id) REFERENCES orders (order_id) ON DELETE CASCADE ON
     UPDATE CASCADE,
    CONSTRAINT fk_payments_customer FOREIGN KEY (customer_id) REFERENCES customers (customer_id) ON DELET
    E CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 8. Table: returns
DROP TABLE IF EXISTS returns;
CREATE TABLE returns (
    return_id           VARCHAR(10)     NOT NULL,
    order_id            VARCHAR(12)     NOT NULL,
    customer_id         VARCHAR(12)     NOT NULL,
    return_date         DATE            NOT NULL,
    reason              VARCHAR(100)    NULL,
    return_amount       DECIMAL(12,2)   NULL,
    refund_status       VARCHAR(20)     DEFAULT 'Completed',
    refund_date         DATE            NULL,
    return_condition    VARCHAR(20)     NULL,
    remarks             TEXT            NULL,
    PRIMARY KEY (return_id),
    INDEX idx_returns_order (order_id),
    INDEX idx_returns_customer (customer_id),
    INDEX idx_returns_reason (reason),
    CONSTRAINT fk_returns_order FOREIGN KEY (order_id) REFERENCES orders (order_id) ON DELETE CASCADE ON
    UPDATE CASCADE,
    CONSTRAINT fk_returns_customer FOREIGN KEY (customer_id) REFERENCES customers (customer_id) ON DELETE
     CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

SET FOREIGN_KEY_CHECKS = 1;


-- ------------------------------------------------------------------------
-- SECTION 2: DATA LOADING TEMPLATES (MYSQL WORKBENCH LOAD DATA INFILE)
-- ------------------------------------------------------------------------
-- Note: Replace '/path/to/csv/' with your actual local file directory.
-- Dates formatted as DD-MM-YYYY in CSV are parsed using STR_TO_DATE(@var, '%d-%m-%Y').

/*
LOAD DATA LOCAL INFILE '/path/to/csv/suppliers DA.csv'
INTO TABLE suppliers
FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS
(supplier_id, supplier_name, contact_person, email, phone, city, state, pincode, category, gstin, payment
    _terms_days, rating, @created_date, is_active)
SET created_date = STR_TO_DATE(@created_date, '%d-%m-%Y');

LOAD DATA LOCAL INFILE '/path/to/csv/products DA.csv'
INTO TABLE products
FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS
```

```
(product_id, product_name, category, subcategory, brand, sku, mrp, selling_price, cost_price, gst_rate, h
    sn_code, weight_grams, supplier_id, rating, review_count, is_active, @launch_date)
SET launch_date = STR_TO_DATE(@launch_date, '%d-%m-%Y');

LOAD DATA LOCAL INFILE '/path/to/csv/inventory DA.csv'
INTO TABLE inventory
FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS
(inventory_id, product_id, warehouse_location, quantity_available, quantity_reserved, reorder_level, reor
    der_quantity, @last_restocked, unit_cost, total_inventory_value, status)
SET last_restocked_date = STR_TO_DATE(@last_restocked, '%d-%m-%Y');

LOAD DATA LOCAL INFILE '/path/to/csv/customers DA.csv'
INTO TABLE customers
FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS
(customer_id, first_name, last_name, email, phone, city, state, pincode, gender, age, segment, @reg_date,
     @login_date, is_verified, is_active, total_orders, total_spent)
SET registration_date = STR_TO_DATE(@reg_date, '%d-%m-%Y'),
    last_login_date = STR_TO_DATE(@login_date, '%d-%m-%Y');

LOAD DATA LOCAL INFILE '/path/to/csv/orders DA.csv'
INTO TABLE orders
FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS
(order_id, customer_id, @ord_date, order_time, status, city, state, pincode, total_amount, gst_amount, sh
    ipping_charge, discount_amount, final_amount, payment_method, shipping_partner, tracking_id, @del_dat
    e, is_cod, channel)
SET order_date = STR_TO_DATE(@ord_date, '%d-%m-%Y'),
    delivered_date = STR_TO_DATE(NULLIF(@del_date, ''), '%d-%m-%Y');

LOAD DATA LOCAL INFILE '/path/to/csv/order_items DA.csv'
INTO TABLE order_items
FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS
(item_id, order_id, product_id, product_name, category, quantity, unit_price, gst_rate, gst_amount, disco
    unt_amount, total_price);

LOAD DATA LOCAL INFILE '/path/to/csv/payments DA.csv'
INTO TABLE payments
FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS
(payment_id, order_id, customer_id, @pay_date, payment_time, payment_method, amount, status, transaction_
    id, bank_name, gateway, refund_amount, @ref_date)
SET payment_date = STR_TO_DATE(@pay_date, '%d-%m-%Y'),
    refund_date = STR_TO_DATE(NULLIF(@ref_date, ''), '%d-%m-%Y');

LOAD DATA LOCAL INFILE '/path/to/csv/returns DA.csv'
INTO TABLE returns
FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS
(return_id, order_id, customer_id, @ret_date, reason, return_amount, refund_status, @ref_date, return_con
    dition, remarks)
SET return_date = STR_TO_DATE(@ret_date, '%d-%m-%Y'),
    refund_date = STR_TO_DATE(NULLIF(@ref_date, ''), '%d-%m-%Y');
*/


-- ------------------------------------------------------------------------
-- SECTION 3: PHASE 1 — DATA QUALITY ASSURANCE & AUDITING QUERIES
-- ------------------------------------------------------------------------

-- 1. Verify Primary Key Uniqueness & Row Counts across all 8 tables
SELECT 'suppliers'   AS table_name, COUNT(*) AS total_rows, COUNT(DISTINCT supplier_id) AS distinct_pk FR
    OM suppliers
UNION ALL
SELECT 'products'    AS table_name, COUNT(*) AS total_rows, COUNT(DISTINCT product_id) AS distinct_pk FRO
    M products
```

```
UNION ALL
SELECT 'inventory'   AS table_name, COUNT(*) AS total_rows, COUNT(DISTINCT inventory_id) AS distinct_pk F
    ROM inventory
UNION ALL
SELECT 'customers'   AS table_name, COUNT(*) AS total_rows, COUNT(DISTINCT customer_id) AS distinct_pk FR
    OM customers
UNION ALL
SELECT 'orders'      AS table_name, COUNT(*) AS total_rows, COUNT(DISTINCT order_id) AS distinct_pk FROM
    orders
UNION ALL
SELECT 'order_items' AS table_name, COUNT(*) AS total_rows, COUNT(DISTINCT item_id) AS distinct_pk FROM o
    rder_items
UNION ALL
SELECT 'payments'    AS table_name, COUNT(*) AS total_rows, COUNT(DISTINCT payment_id) AS distinct_pk FRO
    M payments
UNION ALL
SELECT 'returns'     AS table_name, COUNT(*) AS total_rows, COUNT(DISTINCT return_id) AS distinct_pk FROM
     returns;

-- 2. Audit Referential Integrity: Confirm Zero Orphan Records
SELECT
    (SELECT COUNT(*) FROM orders o LEFT JOIN customers c ON o.customer_id = c.customer_id WHERE c.custome
    r_id IS NULL) AS orphan_orders,
    (SELECT COUNT(*) FROM order_items oi LEFT JOIN orders o ON oi.order_id = o.order_id WHERE o.order_id
    IS NULL) AS orphan_items,
    (SELECT COUNT(*) FROM payments p LEFT JOIN orders o ON p.order_id = o.order_id WHERE o.order_id IS NU
    LL) AS orphan_payments,
    (SELECT COUNT(*) FROM returns r LEFT JOIN orders o ON r.order_id = o.order_id WHERE o.order_id IS NUL
    L) AS orphan_returns;

-- 3. Date Chronology Logic Check: Ensure Delivered Date >= Order Date
SELECT
    COUNT(*) AS total_delivered_orders,
    SUM(CASE WHEN delivered_date < order_date THEN 1 ELSE 0 END) AS invalid_delivery_dates,
    ROUND(AVG(DATEDIFF(delivered_date, order_date)), 2) AS avg_delivery_days,
    MIN(DATEDIFF(delivered_date, order_date)) AS min_delivery_days,
    MAX(DATEDIFF(delivered_date, order_date)) AS max_delivery_days
FROM orders
WHERE delivered_date IS NOT NULL;

-- 4. Audit Order Items Linkage (Flag Synthetic Artifact: 10,071 Orders without Item Rows)
SELECT
    o.status AS order_status,
    COUNT(o.order_id) AS orders_without_items_count,
    ROUND(COUNT(o.order_id) * 100.0 / (SELECT COUNT(*) FROM orders), 2) AS pct_of_total_orders
FROM orders o
LEFT JOIN order_items oi ON o.order_id = oi.order_id
WHERE oi.item_id IS NULL
GROUP BY o.status;

-- 5. Payment Failure Rate Audit
SELECT
    status,
    COUNT(*) AS transaction_count,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM payments), 2) AS pct_share,
    SUM(amount) AS total_amount_inr
FROM payments
GROUP BY status;

-- 6. Returns Distribution by Order Status
SELECT
    o.status AS order_status,
```

```
    COUNT(r.return_id) AS return_requests_logged,
    ROUND(COUNT(r.return_id) * 100.0 / (SELECT COUNT(*) FROM returns), 2) AS pct_of_returns
FROM returns r
JOIN orders o ON r.order_id = o.order_id
GROUP BY o.status
ORDER BY return_requests_logged DESC;

-- 7. Inventory Health Audit (In Stock vs Low Stock vs OOS)
SELECT
    status,
    COUNT(*) AS sku_count,
    SUM(quantity_available) AS total_units,
    SUM(total_inventory_value) AS total_value_inr,
    SUM(CASE WHEN quantity_available < reorder_level THEN 1 ELSE 0 END) AS below_reorder_count
FROM inventory
GROUP BY status;


-- ------------------------------------------------------------------------------
-- SECTION 4: PHASE 2 — EXPLORATORY DATA ANALYSIS (EDA) QUERIES
-- ------------------------------------------------------------------------------

-- Query 1 & 2: Monthly Order Volume & Gross Revenue (GMV) Trend (24 Months)
-- Highlights the Diwali Festive Window (Oct & Nov)
SELECT
    DATE_FORMAT(order_date, '%Y-%m') AS order_month,
    COUNT(order_id) AS total_orders,
    SUM(CASE WHEN status != 'Cancelled' THEN 1 ELSE 0 END) AS active_orders,
    SUM(CASE WHEN status = 'Delivered' THEN 1 ELSE 0 END) AS delivered_orders,
    SUM(CASE WHEN status = 'Cancelled' THEN 1 ELSE 0 END) AS cancelled_orders,
    ROUND(SUM(final_amount) / 10000000.0, 2) AS gross_gmv_cr,
    ROUND(SUM(CASE WHEN status != 'Cancelled' THEN final_amount ELSE 0 END) / 10000000.0, 2) AS active_gm
    v_cr,
    ROUND(SUM(CASE WHEN status = 'Delivered' THEN final_amount ELSE 0 END) / 10000000.0, 2) AS delivered_
    gmv_cr,
    ROUND(AVG(CASE WHEN status != 'Cancelled' THEN final_amount ELSE NULL END), 2) AS active_aov_inr,
    CASE
        WHEN MONTH(order_date) IN (10, 11) THEN 'Diwali Festive Window'
        ELSE 'Regular Window'
    END AS window_classification
FROM orders
GROUP BY DATE_FORMAT(order_date, '%Y-%m'), window_classification
ORDER BY order_month ASC;

-- Query 3: Category-Wise Sales, Units Sold, and Revenue Contribution
SELECT
    oi.category,
    COUNT(DISTINCT oi.order_id) AS distinct_orders,
    SUM(oi.quantity) AS total_units_sold,
    ROUND(SUM(oi.total_price), 2) AS gross_revenue_inr,
    ROUND(SUM(oi.total_price) / 10000000.0, 2) AS gross_revenue_cr,
    ROUND(SUM(oi.total_price) * 100.0 / (SELECT SUM(total_price) FROM order_items), 2) AS revenue_share_p
    ct,
    ROUND(AVG(oi.unit_price), 2) AS avg_unit_selling_price_inr
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
WHERE o.status != 'Cancelled'
GROUP BY oi.category
ORDER BY gross_revenue_inr DESC;

-- Query 4: Order Status Breakdown (Fulfillment Funnel)
SELECT
```

```
    status,
    COUNT(*) AS order_count,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM orders), 2) AS status_share_pct,
    ROUND(SUM(final_amount) / 10000000.0, 2) AS total_value_cr,
    ROUND(AVG(final_amount), 2) AS avg_order_value_inr
FROM orders
GROUP BY status
ORDER BY order_count DESC;

-- Query 5: Top 10 States by Order Volume and Revenue
SELECT
    state,
    COUNT(DISTINCT customer_id) AS unique_customers,
    COUNT(order_id) AS total_active_orders,
    ROUND(COUNT(order_id) * 100.0 / (SELECT COUNT(*) FROM orders WHERE status != 'Cancelled'), 2) AS stat
    e_order_share_pct,
    ROUND(SUM(final_amount) / 10000000.0, 2) AS total_revenue_cr,
    ROUND(AVG(final_amount), 2) AS state_aov_inr
FROM orders
WHERE status != 'Cancelled'
GROUP BY state
ORDER BY total_active_orders DESC
LIMIT 10;

-- Query 6: Customer Segment Distribution (RFM Behavioral Split)
SELECT
    c.segment,
    COUNT(c.customer_id) AS customer_count,
    ROUND(COUNT(c.customer_id) * 100.0 / (SELECT COUNT(*) FROM customers), 2) AS customer_share_pct,
    ROUND(AVG(c.total_orders), 2) AS avg_orders_per_customer,
    ROUND(AVG(c.total_spent), 2) AS avg_clv_inr,
    ROUND(SUM(c.total_spent) / 10000000.0, 2) AS total_segment_spend_cr,
    ROUND(SUM(c.total_spent) * 100.0 / (SELECT SUM(total_spent) FROM customers), 2) AS spend_share_pct
FROM customers c
GROUP BY c.segment
ORDER BY total_segment_spend_cr DESC;

-- Query 7: Payment Method Adoption & Gateway Failure Rates
SELECT
    payment_method,
    COUNT(*) AS total_transactions,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM payments), 2) AS transaction_share_pct,
    ROUND(SUM(amount) / 10000000.0, 2) AS total_volume_cr,
    ROUND(AVG(amount), 2) AS avg_transaction_value_inr,
    SUM(CASE WHEN status = 'Failed' THEN 1 ELSE 0 END) AS failed_transactions,
    ROUND(SUM(CASE WHEN status = 'Failed' THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2) AS failure_rate_pct
FROM payments
GROUP BY payment_method
ORDER BY total_transactions DESC;

-- Query 8: Customer Age Cohorts & Purchase Behavior
SELECT
    CASE
        WHEN age < 25 THEN 'Under 25'
        WHEN age BETWEEN 25 AND 34 THEN '25 - 34 (Prime Demographics)'
        WHEN age BETWEEN 35 AND 44 THEN '35 - 44'
        WHEN age BETWEEN 45 AND 54 THEN '45 - 54'
        ELSE '55 and Above'
    END AS age_cohort,
    COUNT(*) AS customer_count,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM customers), 2) AS customer_pct,
    ROUND(AVG(total_orders), 2) AS avg_orders,
```

```
    ROUND(AVG(total_spent), 2) AS avg_spend_inr
FROM customers
GROUP BY age_cohort
ORDER BY customer_count DESC;

-- Query 9: Return Reasons Distribution & Financial Impact
SELECT
    reason,
    COUNT(*) AS return_count,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM returns), 2) AS return_share_pct,
    ROUND(SUM(return_amount) / 10000000.0, 2) AS total_return_amount_cr,
    ROUND(AVG(return_amount), 2) AS avg_return_value_inr
FROM returns
GROUP BY reason
ORDER BY return_count DESC;

-- Query 10: Average Order Value (AOV) by Customer Segment and Channel
SELECT
    c.segment,
    o.channel,
    COUNT(o.order_id) AS active_orders,
    ROUND(SUM(o.final_amount) / 10000000.0, 2) AS total_revenue_cr,
    ROUND(AVG(o.final_amount), 2) AS aov_inr
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.status != 'Cancelled'
GROUP BY c.segment, o.channel
ORDER BY c.segment, aov_inr DESC;


-- ------------------------------------------------------------------------------
-- SECTION 5: PHASE 3 — 10 CORE BUSINESS KPI CALCULATIONS (BOARD SCORECARD)
-- ------------------------------------------------------------------------------

-- KPI 1: Active GMV & Delivered GMV
SELECT
    '1. Gross Merchandise Value (GMV)' AS metric,
    ROUND(SUM(CASE WHEN status != 'Cancelled' THEN final_amount ELSE 0 END) / 10000000.0, 2) AS active_gm
    v_cr,
    ROUND(SUM(CASE WHEN status = 'Delivered' THEN final_amount ELSE 0 END) / 10000000.0, 2) AS delivered_
    gmv_cr,
    COUNT(CASE WHEN status != 'Cancelled' THEN 1 END) AS active_orders_count,
    COUNT(CASE WHEN status = 'Delivered' THEN 1 END) AS delivered_orders_count
FROM orders;

-- KPI 2: Average Order Value (Active vs Delivered)
SELECT
    '2. Average Order Value (AOV)' AS metric,
    ROUND(AVG(CASE WHEN status != 'Cancelled' THEN final_amount ELSE NULL END), 2) AS active_aov_inr,
    ROUND(AVG(CASE WHEN status = 'Delivered' THEN final_amount ELSE NULL END), 2) AS delivered_aov_inr
FROM orders;

-- KPI 3: Cancellation Rate
SELECT
    '3. Cancellation Rate' AS metric,
    COUNT(*) AS total_orders,
    SUM(CASE WHEN status = 'Cancelled' THEN 1 ELSE 0 END) AS cancelled_orders,
    ROUND(SUM(CASE WHEN status = 'Cancelled' THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2) AS cancellation_ra
    te_pct,
    'Watch (< 8% Target)' AS benchmark_status
FROM orders;
```

```
-- KPI 4: Payment Failure Rate
SELECT
    '4. Payment Failure Rate' AS metric,
    COUNT(*) AS total_payment_attempts,
    SUM(CASE WHEN status = 'Failed' THEN 1 ELSE 0 END) AS failed_attempts,
    ROUND(SUM(CASE WHEN status = 'Failed' THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2) AS failure_rate_pct,
    'Exceeds 2.0% Target' AS benchmark_status
FROM payments;

-- KPI 5: Customer Return Rate (Delivered Orders)
SELECT
    '5. Return Rate on Delivered Orders' AS metric,
    (SELECT COUNT(*) FROM returns) AS total_returns,
    (SELECT COUNT(*) FROM orders WHERE status = 'Delivered') AS delivered_orders,
    ROUND((SELECT COUNT(*) FROM returns) * 100.0 / (SELECT COUNT(*) FROM orders WHERE status = 'Delivered
    '), 2) AS return_rate_on_delivered_pct,
    'High (> 20% Target)' AS benchmark_status;

-- KPI 6: Inventory In-Stock / Fill Rate
SELECT
    '6. Inventory Fill Rate' AS metric,
    COUNT(*) AS total_skus,
    SUM(CASE WHEN status = 'In Stock' THEN 1 ELSE 0 END) AS in_stock_skus,
    SUM(CASE WHEN status = 'Low Stock' THEN 1 ELSE 0 END) AS low_stock_skus,
    SUM(CASE WHEN status = 'Out of Stock' THEN 1 ELSE 0 END) AS oos_skus,
    ROUND(SUM(CASE WHEN status = 'In Stock' THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2) AS in_stock_rate_pc
    t,
    'Below 95% Target' AS benchmark_status
FROM inventory;

-- KPI 7: Electronics Category Revenue Concentration
SELECT
    '7. Category Revenue Concentration' AS metric,
    ROUND(SUM(CASE WHEN category = 'Electronics' THEN total_price ELSE 0 END) * 100.0 / SUM(total_price),
     2) AS electronics_revenue_share_pct,
    'High Risk (> 40% Target)' AS benchmark_status
FROM order_items;

-- KPI 8: Top 5 States Geographic Concentration
WITH ranked_states AS (
    SELECT state, COUNT(*) AS state_orders
    FROM orders
    WHERE status != 'Cancelled'
    GROUP BY state
    ORDER BY state_orders DESC
    LIMIT 5
)
SELECT
    '8. Top 5 States Concentration' AS metric,
    SUM(state_orders) AS top_5_states_orders,
    (SELECT COUNT(*) FROM orders WHERE status != 'Cancelled') AS total_active_orders,
    ROUND(SUM(state_orders) * 100.0 / (SELECT COUNT(*) FROM orders WHERE status != 'Cancelled'), 2) AS to
    p_5_states_share_pct,
    'Concentrated in UP & MH' AS benchmark_status
FROM ranked_states;

-- KPI 9: Digital Channel Performance (Mobile App Dominance)
SELECT
    channel,
    COUNT(*) AS active_orders,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM orders WHERE status != 'Cancelled'), 2) AS channel_sha
    re_pct,
```

```
    ROUND(SUM(final_amount) / 10000000.0, 2) AS revenue_cr,
    ROUND(AVG(final_amount), 2) AS aov_inr
FROM orders
WHERE status != 'Cancelled'
GROUP BY channel
ORDER BY active_orders DESC;

-- KPI 10: Customer Lifetime Value (CLV) by Segment
SELECT
    segment,
    COUNT(customer_id) AS customer_count,
    ROUND(AVG(total_orders), 2) AS avg_orders,
    ROUND(AVG(total_spent), 2) AS avg_clv_inr,
    ROUND(SUM(total_spent) / 10000000.0, 2) AS segment_revenue_cr,
    ROUND(SUM(total_spent) * 100.0 / (SELECT SUM(total_spent) FROM customers), 2) AS spend_share_pct
FROM customers
GROUP BY segment
ORDER BY segment_revenue_cr DESC;


-- ------------------------------------------------------------------------------
-- SECTION 6: ANALYTICAL VIEWS FOR BI DASHBOARDS & EXECUTIVE REPORTING
-- ------------------------------------------------------------------------------

CREATE OR REPLACE VIEW vw_monthly_executive_summary AS
SELECT
    DATE_FORMAT(order_date, '%Y-%m') AS order_month,
    COUNT(order_id) AS total_orders,
    SUM(CASE WHEN status != 'Cancelled' THEN 1 ELSE 0 END) AS active_orders,
    SUM(CASE WHEN status = 'Delivered' THEN 1 ELSE 0 END) AS delivered_orders,
    SUM(CASE WHEN status = 'Cancelled' THEN 1 ELSE 0 END) AS cancelled_orders,
    ROUND(SUM(CASE WHEN status != 'Cancelled' THEN final_amount ELSE 0 END), 2) AS active_gmv,
    ROUND(SUM(CASE WHEN status = 'Delivered' THEN final_amount ELSE 0 END), 2) AS delivered_gmv,
    ROUND(AVG(CASE WHEN status != 'Cancelled' THEN final_amount ELSE NULL END), 2) AS aov,
    ROUND(SUM(CASE WHEN status = 'Cancelled' THEN 1.0 ELSE 0 END) * 100.0 / COUNT(*), 2) AS cancellation_
    rate_pct
FROM orders
GROUP BY DATE_FORMAT(order_date, '%Y-%m');

CREATE OR REPLACE VIEW vw_category_performance_summary AS
SELECT
    oi.category,
    COUNT(DISTINCT oi.order_id) AS orders_count,
    SUM(oi.quantity) AS units_sold,
    ROUND(SUM(oi.total_price), 2) AS total_revenue,
    ROUND(AVG(oi.unit_price), 2) AS avg_unit_price,
    ROUND(SUM(oi.total_price) * 100.0 / (SELECT SUM(total_price) FROM order_items), 2) AS revenue_share_p
    ct
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
WHERE o.status != 'Cancelled'
GROUP BY oi.category;

CREATE OR REPLACE VIEW vw_inventory_alert_summary AS
SELECT
    i.inventory_id,
    p.product_id,
    p.product_name,
    p.category,
    p.brand,
    i.warehouse_location,
    i.quantity_available,
```

```
    i.reorder_level,
    i.reorder_quantity,
    i.status,
    i.total_inventory_value
FROM inventory i
JOIN products p ON i.product_id = p.product_id
WHERE i.status IN ('Low Stock', 'Out of Stock') OR i.quantity_available < i.reorder_level;

-- ================================================================================
-- END OF MYSQL WORKBENCH SCRIPT
-- ================================================================================
```